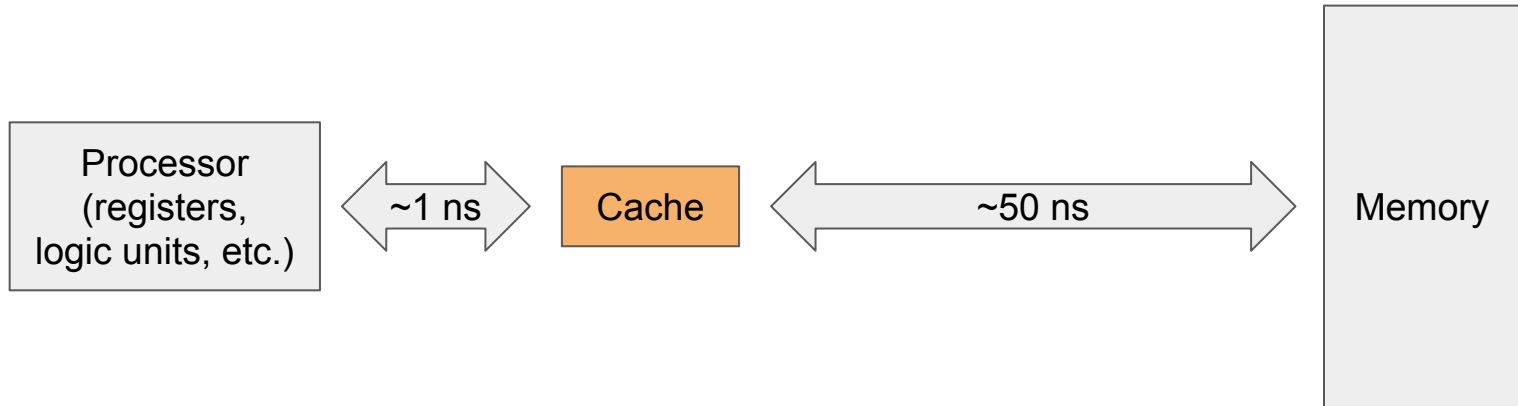


caching!

direct-mapped

Why cache?

- Time improvement
- Accessing information in memory (lw, sw) is expensive
 - Up to 50 cycles of delay (Intel processors)
- Cache is **smaller** and **physically closer** to the processor than memory



Having some data quickly accessible is useful!

When cache?

- Caching everything would defeat the purpose
 - Highlighting/tabbing every page of a textbook
- Temporal Locality
 - After accessing or creating a variable, it is likely to be accessed soon again

```
int accumulation = 0;
for (int i = 0; i<5; i++) {
    for (int j = 0; j<5; j++) {
        accumulation += (i * j);
    }
}
```

Having recently accessed data quickly accessible is useful!

What to cache?

- Spatial Locality

- Array elements stored consecutively in memory
- Often want to iterate through a list of data

```
Arr[5] = {2, 4, 6, 8, 10};  
int sum = 0;  
for (int i = 0; i < 5; i++) {  
    sum += Arr[i];  
}
```

...			
Arr[0]		2	
Arr[1]		4	
Arr[2]		6	
Arr[3]		8	
Arr[4]		10	
...			

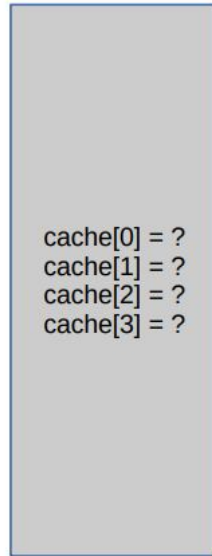
- This loop could take 5 memory accesses OR...

- 1 memory access + 4 cache accesses

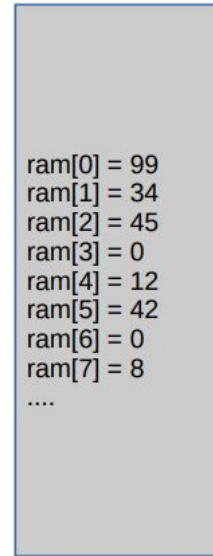
Having nearby elements of memory quickly accessible is useful!

So we want to cache...

But how do we know **where** to cache a certain memory address access?



cache



RAM

What information do we need to figure out?

- Cache structure:
 - How many rows does the cache have?
 - How large are the **blocks** of consecutive data that we store?
- Memory address access that causes us to cache:
 - Which block will we store?
 - What row of the cache will we store the block into?
 - How can we distinguish this block from other blocks of data we might want to store here?

Row	Value
0	?
1	?
2	?
3	?

address	
0	} block 0
1	
2	} block 1
3	
4	} block 2
5	
6	} block 3
7	
8	} block 4
9	
10	} block 5
11	
12	} block 6
13	
14	} block 7
15	

Terms

- Cache structure:
 - How many rows does the cache have? **numrows**
 - How large are the blocks of consecutive data that we store? **blocksize**
- Memory address access that causes us to cache:
 - Which block will we store? **blockid**
 - What row of the cache will we store the block into? **row or index**
 - How can we distinguish this block from other blocks of data we might want to store here? **tag**

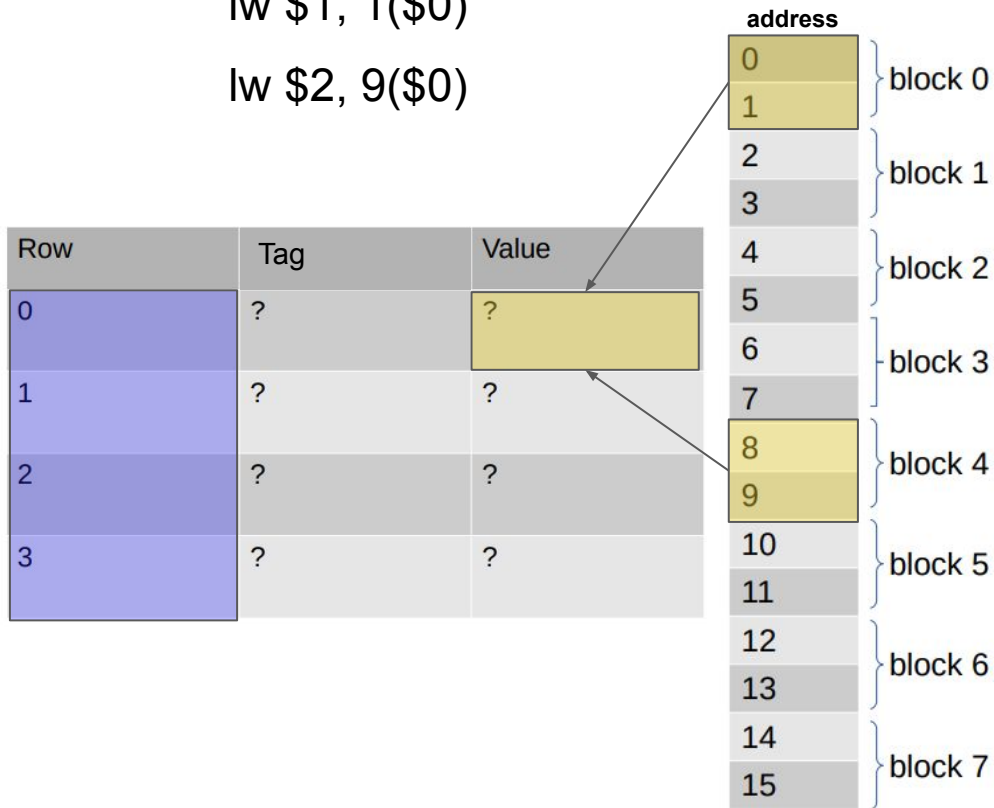
		address	
		0	} block 0
		1	
		2	} block 1
		3	
Row	Value	4	} block 2
		5	
0	?	6	} block 3
1	?	7	
		8	} block 4
		9	
2	?	10	} block 5
3	?	11	
		12	} block 6
		13	
		14	} block 7
		15	

Equations

- Cache structure:
 - *numrows*
 - *blocksize*
- For a memory address:
 - Which block will we store?
 - **$\text{address} // \text{blocksize} = \text{blockid}$**
 - **$1 // 2 = 0$**
 - **$9 // 2 = 4$**
 - What row of the cache will we store the block of data in?
 - **$\text{blockid} \% \text{numrows} = \text{index}$**
 - **$0 \% 4 = 0$**
 - **$4 \% 4 = 0$**
 - How can we distinguish this block from other blocks of data we might want to store here?
 - **$\text{blockid} // \text{numrows} = \text{tag}$**
 - **$0 // 4 = 0$**
 - **$4 // 4 = 1$**

lw \$1, 1(\$0)

lw \$2, 9(\$0)

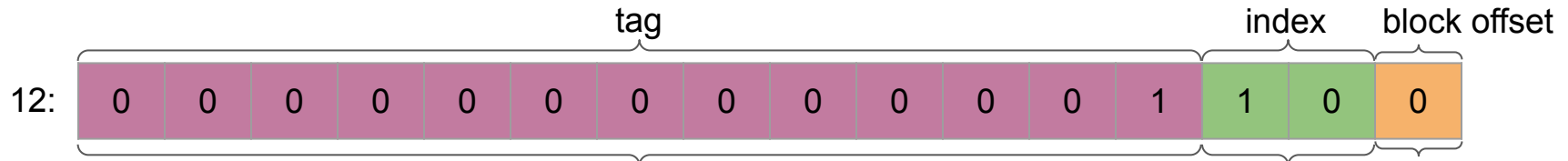


What can we tell from the address?

- Equations:
 - $\text{address} // \text{blocksize} = \text{blockid}$
 - $\text{blockid} \% \text{numrows} = \text{index}$
 - $\text{blockid} // \text{numrows} = \text{tag}$
- Caching value at address 12 in a 4-row direct mapped cache with block size of 2:
 - Blockid = $12 // 2 = 6$
 - Index = $6 \% 4 = 2$
 - Tag = $6 // 4 = 1$
- Block offset:** position of address within the block
 - Observe in the LSBs of the address
 - Number of bits is based on block size

Row	Tag	Value
0	?	?
1	?	?
2	?	?
3	?	?

	address	
0000	0	} block 0
0001	1	
0010	2	} block 1
0011	3	
0100	4	} block 2
0101	5	
0110	6	} block 3
0111	7	
1000	8	} block 4
1001	9	
1010	10	} block 5
1011	11	
1100	12	} block 6
1101	13	
1110	14	} block 7
1111	15	

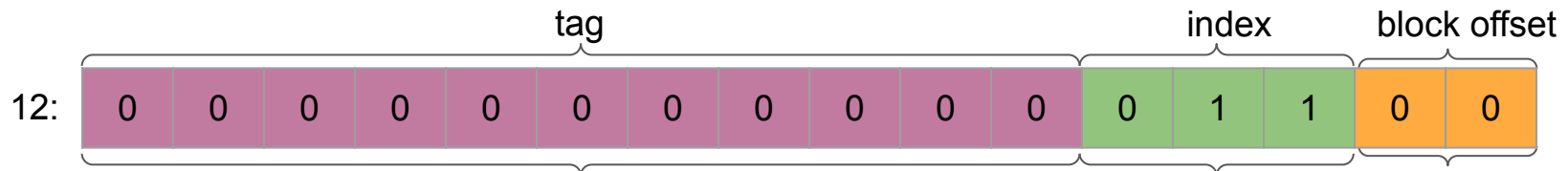


What can we tell from the address?

- The **index** is the next least significant bits after the bit offset
 - Number of bits is based on the number of rows in the cache
 - The **tag** is all the remaining bits of the address
 - Questions:
 - If the block size were 4, how many bits of the address would be used to represent the block offset?
 - If the cache had 8 rows instead of 4, how many bits of the address would be used to represent the index?
- | Row | Tag |
|-----|-----|
| 0 | ? |
| 1 | ? |
| 2 | ? |
| 3 | ? |

Row	Tag	Value
0	?	?
1	?	?
2	?	?
3	?	?

	address	
0000	0	} block 0
0001	1	
0010	2	} block 1
0011	3	
0100	4	} block 2
0101	5	
0110	6	} block 3
0111	7	
1000	8	} block 4
1001	9	
1010	10	} block 5
1011	11	
1100	12	} block 6
1101	13	
1110	14	} block 7
1111	15	



Simulating Direct-Mapped Caches Example **lines = rows*

Given a direct-mapped cache with 4 lines and a blocksize of 8, simulate a cache system accessing the below addresses in sequence.

[85, 106, 88, 124, 125]

numlines = 4
blocksize = 8

blockid = address // 8
index = blockid % 4
tag = blockid // 4

blockid = address // blocksize
index = blockid % numlines
tag = blockid // numlines

Simulating Direct-Mapped Caches Example

[85, 106, 87, 112, 113]

$\text{blockid} = \text{address} // 8$

$\text{index} = \text{blockid} \% 4$

$\text{tag} = \text{blockid} // 4$

Line	V	Tag	Value
0	0		
1	0		
2	0		
3	0		

Simulating Direct-Mapped Caches Example

[85, 106, 87, 112, 113]
M

$\text{blockid} = 85 // 8 = 10$

$\text{index} = 10 \% 4 = 2$

$\text{tag} = 10 // 4 = 2$

Line	V	Tag	Value
0	0		
1	0		
2	1	2	[block 10] Values at 80 - 87
3	0		

Simulating Direct-Mapped Caches Example

[85, **106**, 87, 112, 113]
M M

$\text{blockid} = 106 // 8 = 13$

$\text{index} = 13 \% 4 = 1$

$\text{tag} = 13 // 4 = 3$

Line	V	Tag	Value
0	0		
1	1	3	[block 13] Values at 104 - 111
2	1	2	[block 10] Values at 80 - 87
3	0		

Simulating Direct-Mapped Caches Example

[85, 106, **87**, 112, 113]

M M H

$\text{blockid} = 87 // 8 = 10$

$\text{index} = 10 \% 4 = 2$

$\text{tag} = 10 // 4 = 2$

Line	V	Tag	Value
0	0		
1	1	3	[block 13] Values at 104 - 111
2	1	2	[block 10] Values at 80 - 87
3	0		

Simulating Direct-Mapped Caches Example

[85, 106, 87, **112**, 113]
M M H

$\text{blockid} = 112 // 8 = 14$

$\text{index} = 14 \% 4 = 2$

$\text{tag} = 14 // 4 = 3$

Line	V	Tag	Value
0	0		
1	1	3	[block 13] Values at 104 - 111
2	1	2	[block 10] Values at 80 - 87
3	0		

Simulating Direct-Mapped Caches Example

[85, 106, 87, **112**, 113]
M M H E

$\text{blockid} = 112 // 8 = 14$

$\text{index} = 14 \% 4 = 2$

$\text{tag} = 14 // 4 = 3$

Line	V	Tag	Value
0	0		
1	1	3	[block 13] Values at 104 - 111
2	1	3	[block 14] Values at 112 - 119
3	0		

Simulating Direct-Mapped Caches Example

[85, 106, 87, 112, **113**]

M M H E H

Line	V	Tag	Value
0	0		
1	1	3	[block 13] Values at 104 - 111
2	1	3	[block 14] Values at 112 - 119
3	0		

Simulating Direct-Mapped Caches Example

[85, 106, 87, 112, **113**]

M M H E H

Accessing the cache is required for **every** memory access, even if it's a miss!

Line	V	Tag	Value
0	0		
1	1	3	[block 13] Values at 104 - 111
2	1	3	[block 14] Values at 112 - 119
3	0		